Comprehensive Java Notes: From Basics to Advanced

Table of Contents

1. Introduction to Java
2. Java Basics: The Building Blocks
3. Object-Oriented Programming (OOP) in Java
4. Core Java APIs: Essential Libraries
5. Exception Handling
6. Concurrency (Multithreading)
7. Generics
8. Lambda Expressions & Stream API (Java 8+)
9. Date and Time API (Java 8+)
10. Advanced Java Topics
    - Input/Output (I/O) Streams
    - Networking
    - Java Database Connectivity (JDBC)
    - Reflection
    - Annotations
    - JVM Internals & Garbage Collection
    - Java Platform Module System (JPMS - Java 9+)
    - Design Patterns
11. Modern Java Features (Java 14+)
12. Where to Go Next

---

1. Introduction to Java

- **What is Java?**
    - A high-level, object-oriented, class-based, concurrent, secured, and general-purpose computer-programming language.
    - Developed by James Gosling at Sun Microsystems (now Oracle).
    - First released in 1995.
- **Key Principles:**
    - **"Write Once, Run Anywhere" (WORA):** Java code is compiled into bytecode (.class files), which can run on any device with a Java Virtual Machine (JVM).

    - **Platform Independent:** Not tied to a specific operating system or hardware.
    - **Object-Oriented:** Everything in Java is an object (almost).
    - **Simple:** Designed to be easy to learn and use.
    - **Secure:** Built-in security features for network-distributed applications.
    - **Robust:** Strong memory management, exception handling, and garbage collection.
    - **Multithreaded:** Supports concurrent execution of multiple parts of a program.
    - **High Performance:** Just-In-Time (JIT) compilers enable high performance.
    - **Distributed:** Designed for distributed environments of the internet.
    - **Dynamic:** Adapts to evolving environments.
- **Java Development Kit (JDK):**
    - **JRE (Java Runtime Environment):** Contains the JVM and standard libraries required to *run* Java applications.
    - **JVM (Java Virtual Machine):** The runtime engine that executes Java bytecode. It's an abstract machine.
    - **Development Tools:** Compiler (javac), debugger (jdb), archiver (jar), etc. (needed to *develop* Java applications).

---

2. Java Basics: The Building Blocks

2.1. Basic Syntax and Structure

- **main Method:** The entry point of any Java application.

Java

```
public class MyFirstApp {
   public static void main(String[] args) {
      System.out.println("Hello, Java!");
   }
}
```

- **Comments:**
    - Single-line: // This is a comment
    - Multi-line: /* This is a multi-line comment */
    - Documentation: /** This is for Javadoc */
- **Keywords:** Reserved words (e.g., public, class, static, void, int, if, else, for, while, new, this, super, try, catch, finally).

2.2. Variables and Data Types

- **Variables:** Named memory locations to store data.
- **Data Types:**
    - **Primitive Data Types:** Store simple values directly.
        - byte (1 byte, -128 to 127)
        - short (2 bytes)
        - int (4 bytes, default for whole numbers)
        - long (8 bytes)
        - float (4 bytes, single-precision floating-point)
        - double (8 bytes, double-precision floating-point, default for decimals)
        - boolean (1 bit, true or false)
        - char (2 bytes, single Unicode character)
    - **Non-Primitive (Reference) Data Types:** Store references to objects.
        - String (a class for text)
        - Arrays
        - Classes, Interfaces
- **Variable Declaration and Initialization:**

Java

```
int age = 30;
String name = "Alice";
double price = 19.99;
boolean isActive = true;
char grade = 'A';
```

- **Type Casting:** Converting one data type to another.
    - **Widening (Implicit):** Smaller to larger type (e.g., int to long).
    - **Narrowing (Explicit):** Larger to smaller type (requires cast, e.g., double to int).

Java

```
int myInt = (int) 3.14; // Narrowing
long myLong = 100;     // Widening
```

2.3. Operators

- **Arithmetic:** +, -, *, /, % (modulus)

- **Assignment:** =, +=, -=, *=, /=, %=
- **Comparison (Relational):** ==, !=, >, <, >=, <= (return boolean)
- **Logical:** && (AND), || (OR), ! (NOT)
- **Increment/Decrement:** ++, --
- **Bitwise:** &, |, ^, ~, <<, >>, >>>
- **Ternary:** condition ? expression1 : expression2;

2.4. Control Flow Statements

- **Conditional Statements:**
    - if-else if-else: Executes code based on conditions.
    - switch: Selects one of many code blocks to be executed.

Java

```
if (score >= 90) {
   System.out.println("A");
} else if (score >= 80) {
   System.out.println("B");
} else {
   System.out.println("C");
}


switch (day) {
   case 1: System.out.println("Monday"); break;
   case 7: System.out.println("Sunday"); break;
   default: System.out.println("Other day");
}
```

- **Looping Statements:**
    - for: Repeats a block of code a specific number of times.
    - while: Repeats a block of code as long as a condition is true.
    - do-while: Repeats a block of code at least once, then as long as a condition is true.
    - for-each (Enhanced for loop): Iterates over arrays or collections.

Java

```
for (int i = 0; i < 5; i++) { /* ... */ }
```

```
int i = 0;

while (i < 5) { /* ... */ i++; }

do { /* ... */ i++; } while (i < 5);

for (String item : items) { /* ... */ }
```

- **Jump Statements:**
    - break: Exits a loop or switch statement.
    - continue: Skips the current iteration of a loop.
    - return: Exits a method and optionally returns a value.

---

3. Object-Oriented Programming (OOP) in Java

Java is fundamentally an object-oriented language. OOP principles help organize code, making it reusable, maintainable, and scalable.

3.1. Classes and Objects

- **Class:** A blueprint or template for creating objects. It defines the properties (attributes/fields) and behaviors (methods) that objects of that class will have.
- **Object:** An instance of a class. A real-world entity with state (data) and behavior (methods).
- **Declaration & Instantiation:**

Java

```
class Dog {
   String name; // field/attribute
   int age;     // field/attribute

   void bark() { // method/behavior
      System.out.println(name + " says Woof!");
   }
}

// Creating an object (instance)
Dog myDog = new Dog();
myDog.name = "Buddy";
myDog.age = 5;
myDog.bark(); // Calling a method
```

- **Constructors:** Special methods used to initialize objects. They have the same name as the class and no return type.
    - **Default Constructor:** Provided automatically if no other constructor is defined.
    - **Parameterized Constructor:** Takes arguments to initialize fields.

Java

```
class Dog {
  String name;
  int age;
  public Dog(String name, int age) { // Parameterized Constructor
    this.name = name;
    this.age = age;
  }
}
Dog anotherDog = new Dog("Lucy", 2);
```

- **this keyword:** Refers to the current object. Used to distinguish instance variables from parameters with the same name.

3.2. Four Pillars of OOP

3.2.1. Encapsulation

- **Definition:** Bundling data (fields) and methods that operate on the data within a single unit (class). It also involves restricting direct access to some of an object's components, typically done via access modifiers.
- **Access Modifiers:**
    - public: Accessible from anywhere.
    - protected: Accessible within the same package and by subclasses.
    - default (no keyword): Accessible only within the same package.
    - private: Accessible only within the same class.
- **Getters and Setters:** Public methods (also known as accessors and mutators) to access and modify private fields.

Java

```
class Person {
  private String name; // Encapsulated field

  public String getName() { // Getter
```

```
        return name;
    }
    public void setName(String name) { // Setter
        this.name = name;
    }
}
```

3.2.2. Inheritance

- **Definition:** A mechanism where one class (subclass/child class) acquires the properties and behaviors of another class (superclass/parent class). Promotes code reusability.
- **extends keyword:** Used to establish inheritance.
- **Method Overriding:** A subclass provides a specific implementation for a method that is already defined in its superclass.
- **super keyword:** Used to refer to the immediate parent class instance. Can be used to call parent class constructor or methods.
- **final keyword:**
    - final class: Cannot be subclassed.
    - final method: Cannot be overridden.
    - final variable: Its value cannot be changed after initialization.
- **Object class:** The root class for all Java classes (implicitly extends Object).

Java

```
class Animal { // Superclass
    void eat() { System.out.println("Animal eats"); }
}


class Cat extends Animal { // Subclass
    void meow() { System.out.println("Cat meows"); }
    @Override
    void eat() { System.out.println("Cat eats fish"); } // Method Overriding
}
```

3.2.3. Polymorphism

- **Definition:** "Many forms." The ability of an object to take on many forms. In Java, it typically means a single interface can be used for different data types.

- **Compile-time Polymorphism (Method Overloading):**
    - Methods with the same name but different parameters (number, type, or order).
- **Run-time Polymorphism (Method Overriding):**
    - Achieved through inheritance and interfaces. The actual method called is determined at runtime based on the object's actual type.
    - **Upcasting:** A superclass reference variable referring to a subclass object.

Java

```
class Shape {
   void draw() { System.out.println("Drawing a shape"); }
}
class Circle extends Shape {
   void draw() { System.out.println("Drawing a circle"); }
}
class Rectangle extends Shape {
   void draw() { System.out.println("Drawing a rectangle"); }
}


public class PolyDemo {
   public static void main(String[] args) {
      Shape s1 = new Circle();     // Upcasting
      Shape s2 = new Rectangle();  // Upcasting
      s1.draw(); // Calls Circle's draw() at runtime
      s2.draw(); // Calls Rectangle's draw() at runtime
   }
}
```

3.2.4. Abstraction

- **Definition:** Hiding the implementation details and showing only the necessary functionality to the user.
- **Abstract Class:**
    - A class declared with the abstract keyword.
    - Can have abstract (no implementation) and non-abstract (concrete) methods.
    - Cannot be instantiated directly; requires a subclass to be instantiated.

    - If a class has at least one abstract method, it must be declared abstract.
- **Interface:**
    - A blueprint of a class.
    - Contains abstract methods (implicitly public abstract before Java 8).
    - Can contain public static final fields (constants).
    - A class implements an interface using the implements keyword.
    - Starting Java 8, interfaces can have default and static methods.
    - Starting Java 9, interfaces can have private methods.
    - Supports multiple inheritance (of types, not implementation).

Java

```
abstract class Vehicle { // Abstract Class
   abstract void start(); // Abstract method
   void stop() { System.out.println("Vehicle stopped"); }
}

class Car extends Vehicle {
   @Override
   void start() { System.out.println("Car started with key"); }
}

interface Flyable { // Interface
   void fly(); // Implicitly public abstract
   default void landing() { // Default method (Java 8+)
      System.out.println("Landing gracefully.");
   }
}

class Bird implements Flyable {
   @Override
   public void fly() { System.out.println("Bird is flying"); }
}
```

---

4. Core Java APIs: Essential Libraries

4.1. String Class

- **Immutability:** String objects are immutable. Once created, their value cannot be changed. Any operation that seems to modify a String actually creates a new String object.
- **String vs. StringBuilder vs. StringBuffer:**
    - String: Immutable, good for constant text.
    - StringBuilder: Mutable, not thread-safe (faster for single-threaded environment).
    - StringBuffer: Mutable, thread-safe (slower but safe for multithreaded environment).
- **Common Methods:** length(), charAt(), substring(), indexOf(), equals(), equalsIgnoreCase(), startsWith(), endsWith(), toUpperCase(), toLowerCase(), trim(), replace(), concat().

4.2. Wrapper Classes

- Primitive data types (like int, char, boolean) are not objects. Wrapper classes provide object representations for them.
- Used in Collections, where only objects can be stored.
- **Autoboxing:** Automatic conversion of primitive to wrapper object (e.g., int to Integer).
- **Unboxing:** Automatic conversion of wrapper object to primitive (e.g., Integer to int).
- **Examples:** Integer, Double, Boolean, Character, Long, Float, Short, Byte.

Java

```
int num = 10;

Integer obj = num; // Autoboxing

int anotherNum = obj; // Unboxing
```

4.3. Arrays

- **Definition:** A fixed-size, ordered collection of elements of the same data type.
- **Declaration & Initialization:**

Java

```
int[] numbers = new int[5]; // Declaration and instantiation

numbers[0] = 10;

int[] primes = {2, 3, 5, 7, 11}; // Declaration and initialization
```

- **Multidimensional Arrays:** Arrays of arrays.

Java

```
int[][] matrix = new int[3][3];
```

- **Arrays class:** Provides utility methods for arrays (e.g., sort(), binarySearch(), equals(), toString()).

4.4. The Collections Framework

- A set of interfaces and classes for representing and manipulating collections of objects.
- **Core Interfaces:**
    - Collection: Root interface for all collection types.
    - List: Ordered collection (sequence). Allows duplicate elements. (e.g., ArrayList, LinkedList, Vector).
    - Set: Unordered collection. Does not allow duplicate elements. (e.g., HashSet, LinkedHashSet, TreeSet).
    - Queue: Designed for holding elements prior to processing. Follows FIFO (First-In, First-Out). (e.g., PriorityQueue, ArrayDeque).
    - Map: Maps unique keys to values. (e.g., HashMap, LinkedHashMap, TreeMap, Hashtable).
- **Common Implementations:**
    - ArrayList: Resizable array implementation of the List interface. Fast random access.
    - LinkedList: Doubly-linked list implementation of List and Deque. Efficient insertions/deletions.
    - HashSet: Uses a hash table for storage. Fast operations, no guaranteed order.
    - TreeSet: Stores elements in a sorted order (natural order or custom comparator).
    - HashMap: Hash table-based implementation of the Map interface. Fast key-value lookups.
    - TreeMap: Stores key-value pairs in a sorted order of keys.
- **Collections class:** Provides static methods for operating on collections (e.g., sort(), reverse(), shuffle(), max()).

---

5. Exception Handling

- **What are Exceptions?** Abnormal events that disrupt the normal flow of a program.
- **Error vs. Exception:**
    - **Error:** Irrecoverable conditions (e.g., OutOfMemoryError, StackOverflowError). Application typically terminates.
    - **Exception:** Can be caught and handled.
- **Types of Exceptions:**

    - **Checked Exceptions:** Checked at compile-time. Must be caught or declared (throws). (e.g., IOException, SQLException).
    - **Unchecked Exceptions (Runtime Exceptions):** Not checked at compile-time. Occur at runtime. (e.g., NullPointerException, ArrayIndexOutOfBoundsException, ArithmeticException). Do not *have* to be caught or declared.
- **try-catch-finally Block:**
    - try: Contains code that might throw an exception.
    - catch: Catches and handles a specific type of exception. Can have multiple catch blocks.
    - finally: Contains code that *always* executes, regardless of whether an exception occurred or was caught. Used for cleanup.

Java

```
try {
   int result = 10 / 0; // ArithmeticException
} catch (ArithmeticException e) {
   System.out.println("Cannot divide by zero: " + e.getMessage());
} finally {
   System.out.println("Execution finished.");
}
```

- **throw keyword:** Used to explicitly throw an exception from within a method.
- **throws keyword:** Used in a method signature to declare that a method might throw certain checked exceptions.
- **Custom Exceptions:** You can create your own exception classes by extending Exception (for checked) or RuntimeException (for unchecked).
- **try-with-resources (Java 7+):** Ensures that resources (like file streams) are automatically closed after the try block, even if an exception occurs. Resources must implement AutoCloseable.

Java

```
try (BufferedReader reader = new BufferedReader(new FileReader("file.txt"))) {
   String line;
   while ((line = reader.readLine()) != null) {
      System.out.println(line);
   }
} catch (IOException e) {
```

```
    System.err.println("Error reading file: " + e.getMessage());
}
```

---

6. Concurrency (Multithreading)

- **Process vs. Thread:**
  - **Process:** An independent execution unit with its own memory space.
  - **Thread:** A lightweight sub-process within a process. Shares memory space with other threads of the same process.
- **Benefits of Multithreading:** Improved responsiveness, faster execution, better resource utilization.
- **Creating Threads:**

1. **Extending Thread class:**

Java

```
class MyThread extends Thread {
    public void run() {
        System.out.println("Thread running: " + Thread.currentThread().getName());
    }
}
MyThread t = new MyThread();
t.start(); // Invokes run()
```

2. **Implementing Runnable interface (Recommended):**

Java

```
class MyRunnable implements Runnable {
    public void run() {
        System.out.println("Runnable running: " + Thread.currentThread().getName());
    }
}
Thread t = new Thread(new MyRunnable());
t.start();
```

- **Why Runnable is preferred:** Java doesn't support multiple inheritance of classes. By implementing Runnable, your class can still extend another class.

- **Thread Lifecycle:** New, Runnable, Running, Blocked/Waiting, Timed Waiting, Terminated.

- **Thread Synchronization:** Mechanisms to control access to shared resources by multiple threads to prevent data corruption.
    - **synchronized keyword:** Used on methods or blocks to ensure only one thread can execute it at a time for a given object.
    - **wait(), notify(), notifyAll():** Methods (from Object class) used for inter-thread communication. Must be called within a synchronized block/method.
- **java.util.concurrent package (Advanced Concurrency):**
    - **Executors:** ExecutorService and Executors factory class for managing thread pools.
    - **Future and Callable:** For tasks that return a result and/or throw exceptions.
    - **Locks:** ReentrantLock for more flexible locking than synchronized.
    - **Semaphores:** Control access to a set of resources.
    - **Atomic Variables:** (e.g., AtomicInteger) for thread-safe operations on single variables without explicit locking.
    - **Concurrent Collections:** (e.g., ConcurrentHashMap, CopyOnWriteArrayList) thread-safe alternatives to standard collections.

---

7. Generics

- **What are Generics?** A feature that allows you to write classes, interfaces, and methods that operate on types as parameters.
- **Benefits:**
    - **Type Safety:** Catches type-related errors at compile-time instead of runtime.
    - **Code Reusability:** Write a single piece of code that works with different types.
    - **Elimination of Casts:** Reduces the need for explicit type casting.
- **Generic Classes:**

Java

```
class Box<T> { // T is a type parameter
   private T content;
   public Box(T content) { this.content = content; }
   public T getContent() { return content; }
}
Box<Integer> intBox = new Box<>(123);
Box<String> stringBox = new Box<>("Hello");
```

- **Generic Methods:**

Java

```
public <T> void printArray(T[] array) {
    for (T element : array) {
        System.out.print(element + " ");
    }
    System.out.println();
}
```

- **Bounded Type Parameters:** Restrict the types that can be used for a type parameter.
  - extends: Upper bound (e.g., <T extends Number> means T must be Number or its subclass).
  - super: Lower bound (e.g., <T super Integer> means T must be Integer or its superclass).
- **Wildcards (?):**
  - ? extends Type: Upper-bounded wildcard (read-only operations).
  - ? super Type: Lower-bounded wildcard (write-only operations).
  - ?: Unbounded wildcard (any type).
- **Type Erasure:** Generics information is present only at compile time; it is "erased" and replaced with raw types (e.g., Object) at runtime for backward compatibility.

---

8. Lambda Expressions & Stream API (Java 8+)

8.1. Lambda Expressions

- **What are Lambdas?** A concise way to represent an anonymous function (a function without a name).
- **Syntax:** (parameters) -> expression or (parameters) -> { statements; }
- **Functional Interface:** An interface with exactly one abstract method. Lambdas can only be used to implement functional interfaces.
  - Annotated with @FunctionalInterface.
  - Examples: Runnable, Comparator, Callable, Consumer, Predicate, Function, Supplier.
- **Benefits:** Enables functional programming style, makes code more readable for simple operations, suitable for callbacks.

Java

```
// Traditional anonymous inner class
Runnable oldRunnable = new Runnable() {
```

```
    @Override
    public void run() { System.out.println("Running old way"); }
};

// Lambda equivalent
Runnable newRunnable = () -> System.out.println("Running new way");

// Using with Collections.sort
List<String> names = Arrays.asList("Alice", "Bob", "Charlie");
Collections.sort(names, (s1, s2) -> s1.compareTo(s2)); // Lambda as Comparator
```

8.2. Stream API

- **What are Streams?** A sequence of elements from a source that supports aggregate operations. They don't store data; they process data.
- **Source:** Can be a collection, array, I/O channel, etc.
- **Operations:**
    - **Intermediate Operations:** Return another stream. They are *lazy* (don't execute until a terminal operation is called). Examples: filter(), map(), flatMap(), distinct(), sorted(), limit(), skip().
    - **Terminal Operations:** Produce a result or a side-effect. They *trigger* the execution of intermediate operations. Examples: forEach(), collect(), reduce(), count(), min(), max(), sum(), average(), anyMatch(), allMatch(), noneMatch(), findFirst(), findAny().
- **Pipelining:** Chaining multiple stream operations.
- **Parallel Streams:** Can leverage multiple CPU cores for faster processing (.parallelStream()).
- **Example:**

Java

```
List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5, 6, 7, 8, 9, 10);

// Filter even numbers, double them, and collect into a new list
List<Integer> doubledEvens = numbers.stream()
                    .filter(n -> n % 2 == 0) // Intermediate
                    .map(n -> n * 2)        // Intermediate
                    .collect(Collectors.toList()); // Terminal
```

```
// Sum of odd numbers
int sumOfOdds = numbers.stream()
                .filter(n -> n % 2 != 0)
                .reduce(0, Integer::sum); // Terminal with method reference
```

- **Method References:** A shorthand for lambda expressions that call an existing method.
    - Static method: ClassName::staticMethodName
    - Instance method of a particular object: object::instanceMethodName
    - Instance method of an arbitrary object of a particular type: ClassName::instanceMethodName
    - Constructor: ClassName::new

---

9. Date and Time API (Java 8+)

- **Old API (java.util.Date, java.util.Calendar):** Problematic (mutable, not thread-safe, poor design, hard to use).
- **New API (java.time package):** Inspired by Joda-Time. Immutable, thread-safe, clearly defined, and easier to use.
- **Key Classes:**
    - LocalDate: Date without time-zone (e.g., 2025-06-09).
    - LocalTime: Time without date or time-zone (e.g., 13:30:45).
    - LocalDateTime: Date and time without time-zone.
    - ZonedDateTime: Date and time with a time-zone.
    - Instant: A point in time (machine readable, epoch seconds).
    - Duration: A quantity of time (seconds, nanoseconds).
    - Period: A quantity of time based on dates (years, months, days).
    - DateTimeFormatter: For formatting and parsing date/time objects.
- **Examples:**

Java

```
LocalDate today = LocalDate.now(); // Current date
LocalTime now = LocalTime.now();   // Current time
LocalDateTime currentDateTime = LocalDateTime.now();


LocalDate specificDate = LocalDate.of(2023, 10, 26);
```

```
LocalDateTime birthday = LocalDateTime.of(1990, 5, 15, 10, 30);

// Adding/Subtracting
LocalDate nextWeek = today.plusWeeks(1);
LocalTime tenMinutesLater = now.plusMinutes(10);

// Calculating difference
Period age = Period.between(LocalDate.of(1990, 1, 1), today);
System.out.println("Age: " + age.getYears() + " years");

// Formatting
DateTimeFormatter formatter = DateTimeFormatter.ofPattern("dd-MM-yyyy HH:mm");
String formattedDateTime = currentDateTime.format(formatter);
System.out.println(formattedDateTime);
```

---

10. Advanced Java Topics

10.1. Input/Output (I/O) Streams

- **What are Streams?** A sequence of data.
    - **Input Stream:** Reads data from a source (e.g., file, network).
    - **Output Stream:** Writes data to a destination.
- **Byte Streams:** Handle raw bytes (InputStream, OutputStream). Used for binary data (images, audio).
    - FileInputStream, FileOutputStream
- **Character Streams:** Handle characters (Reader, Writer). Used for text data.
    - FileReader, FileWriter
    - BufferedReader, BufferedWriter (for buffered I/O, better performance)
- **Serializable Interface:** Marks an object so it can be converted into a byte stream (for saving to file or sending over network).
- **File class:** Represents files and directories.
- **NIO (New I/O) / NIO.2 (Java 7+):** Provides more advanced, non-blocking I/O operations and better file system interaction.
    - Path, Paths, Files classes (much preferred for file operations).

Java

```
// Reading from a file (NIO.2 example)
Path filePath = Paths.get("data.txt");
try {
    List<String> lines = Files.readAllLines(filePath);
    lines.forEach(System.out::println);
} catch (IOException e) {
    e.printStackTrace();
}

// Writing to a file (NIO.2 example)
Path outputPath = Paths.get("output.txt");
String content = "Hello from Java!";
try {
    Files.write(outputPath, content.getBytes());
} catch (IOException e) {
    e.printStackTrace();
}
```

10.2. Networking (java.net)

- **Socket Programming:** Enables communication between applications over a network.
    - **Socket:** Client-side endpoint for connection.
    - **ServerSocket:** Server-side endpoint for listening to incoming connections.
- **URLs and URIs:**
    - URL class for interacting with URLs (Uniform Resource Locators).
- **HTTP Client (Java 11+):** Modern API for sending HTTP requests and handling responses (replaces older HttpURLConnection).

10.3. Java Database Connectivity (JDBC)

- **What is JDBC?** A Java API that defines how a Java application can access a database.
- **Driver:** Database-specific software that implements the JDBC API.
- **Steps to Connect to a Database:**
    1. Load the JDBC driver (often not needed explicitly with modern drivers).

2. Establish a connection (Connection object).
3. Create a statement (Statement or PreparedStatement object).
4. Execute a query (executeQuery() for SELECT, executeUpdate() for INSERT/UPDATE/DELETE).
5. Process the result set (ResultSet object).
6. Close resources (Result Set, Statement, Connection).

- **PreparedStatement:** Preferred over Statement for parameterized queries (prevents SQL injection and improves performance).
- **Connection Pooling:** Managing a pool of database connections to improve performance and resource utilization.

10.4. Reflection (java.lang.reflect)

- **What is Reflection?** The ability of a program to examine or modify its own structure and behavior at runtime.
- **Uses:**
  - Inspect classes, interfaces, fields, and methods at runtime.
  - Instantiate objects and invoke methods dynamically.
  - Used by frameworks (Spring, Hibernate, JUnit) extensively.
- **Key Classes:** Class, Constructor, Method, Field.

Java

```
// Example: Getting class name and methods using Reflection
try {
   Class<?> myClass = Class.forName("java.lang.String");
   System.out.println("Class Name: " + myClass.getName());
   Method[] methods = myClass.getMethods();
   for (Method method : methods) {
      System.out.println("Method: " + method.getName());
   }
} catch (ClassNotFoundException e) {
   e.printStackTrace();
}
```

10.5. Annotations

- **What are Annotations?** Metadata that can be added to Java source code. They don't directly affect program execution but provide information to compilers, JVM, or development tools.

- **Types:**
    - **Built-in Annotations:** @Override, @Deprecated, @SuppressWarnings, @FunctionalInterface.
    - **Custom Annotations:** You can define your own.
- **Uses:** Configuration (Spring, Hibernate), code analysis (Lombok), unit testing (JUnit), code generation.

10.6. JVM Internals & Garbage Collection

- **JVM Architecture:**
    - **Classloader Subsystem:** Loads, links, and initializes classes.
    - **Runtime Data Areas:**
        - **Method Area:** Stores class structure, metadata, static variables, code.
        - **Heap:** Runtime data area for objects. This is where garbage collection happens.
        - **Stack:** Stores local variables, partial results, and method calls (one stack frame per method call).
        - **PC Registers:** Stores the address of the JVM instruction currently being executed.
        - **Native Method Stacks:** For native methods.
    - **Execution Engine:** Executes bytecode.
        - **Interpreter:** Executes bytecode instruction by instruction.
        - **JIT (Just-In-Time) Compiler:** Compiles frequently executed bytecode into native machine code for faster execution.
        - **Garbage Collector:** Automatically manages memory by reclaiming space from unreferenced objects.
- **Garbage Collection (GC):**
    - **Automatic Memory Management:** JVM automatically frees up memory occupied by objects that are no longer referenced.
    - **Generational Hypothesis:** Most objects are short-lived, and a few objects are long-lived. This leads to generational GC (Young Generation, Old Generation, Permanent Generation/Metaspace).
    - **GC Algorithms:** Serial, Parallel, CMS (Concurrent Mark Sweep), G1 (Garbage First), Shenandoah, ZGC. Each has different performance characteristics and target use cases.

10.7. Java Platform Module System (JPMS - Java 9+)

- **Project Jigsaw:** The effort to modularize the JDK itself and provide a module system for Java applications.

- **module-info.java:** The module descriptor file.
- **Key Concepts:**
    - **Modules:** Named, self-describing components containing code and resources.
    - **exports:** Declares which packages are visible to other modules.
    - **requires:** Declares dependencies on other modules.
    - **uses:** Declares a dependency on a service interface.
    - **provides...with:** Declares a service implementation.
- **Benefits:** Strong encapsulation, reliable configuration, improved performance, easier scalability.

10.8. Design Patterns

- **What are Design Patterns?** Reusable solutions to common problems in software design. Not code, but general concepts.
- **Categories:**
    - **Creational Patterns:** (e.g., Singleton, Factory Method, Abstract Factory, Builder, Prototype) - how objects are created.
    - **Structural Patterns:** (e.g., Adapter, Bridge, Composite, Decorator, Facade, Flyweight, Proxy) - how classes and objects are composed to form larger structures.
    - **Behavioral Patterns:** (e.g., Chain of Responsibility, Command, Interpreter, Iterator, Mediator, Memento, Observer, State, Strategy, Template Method, Visitor) - how objects interact and distribute responsibilities.
- **Understanding and applying them is crucial for building robust and maintainable enterprise applications.**

---

11. Modern Java Features (Java 14+)

Java has a rapid release cycle (every 6 months) bringing new features frequently.

- **Records (Java 16):** Concise way to declare immutable data classes.

Java

```
public record Point(int x, int y) {} // Automatically provides constructor, getters, equals(), hashCode(), toString()
```

- **Sealed Classes (Java 17):** Restrict which other classes or interfaces can extend or implement them. Enhances control over class hierarchies.

Java

```
public abstract sealed class Shape permits Circle, Square {}

final class Circle extends Shape {}
```

```
final class Square extends Shape {}
```

- **Pattern Matching for instanceof (Java 16):** Reduces boilerplate for type casting.

Java

```
// Old way
if (obj instanceof String) {
   String s = (String) obj;
   System.out.println(s.length());
}

// New way (Pattern Matching)
if (obj instanceof String s) { // 's' is automatically cast here
   System.out.println(s.length());
}
```

- **Text Blocks (Java 15):** Multiline string literals without the need for escape sequences.

Java

```
String json = """
   {
      "name": "Alice",
      "age": 30
   }
   """;
```

- **Switch Expressions (Java 14):** Returns a value from a switch statement and avoids fall-through issues.

Java

```
String dayType = switch (day) {
   case MONDAY, FRIDAY -> "Workday";
   case SATURDAY, SUNDAY -> "Weekend";
   default -> "Midweek";
};
```

- **Foreign Function & Memory API (Preview):** Modern replacement for JNI (Java Native Interface) for interacting with native code.

- **Virtual Threads (Project Loom - Java 21+):** Lightweight threads managed by the JVM, significantly improving concurrency for I/O-bound applications.

---

12. Where to Go Next

- **Practice, Practice, Practice:** The most crucial step. Solve coding challenges, build small projects.
- **Explore Frameworks:**
    - **Spring / Spring Boot:** For building enterprise applications (web, microservices).
    - **Hibernate / JPA:** For Object-Relational Mapping (ORM) and database interaction.
    - **JUnit / Mockito:** For unit and integration testing.
- **Web Development:** Learn about Servlets, JSPs, and then move to modern frameworks like Spring MVC, Spring WebFlux.
- **Database Interaction:** Deeper dive into JDBC, ORM tools.
- **Cloud Development:** Microservices, Docker, Kubernetes, cloud platforms (AWS, Azure, GCP).
- **Specific Domains:** Android development, Big Data (Hadoop, Spark), etc.
- **Stay Updated:** Follow Java news, new language features, and best practices.